GEMAMX v2

Pipeline de Validación Farmacológica Computacional

Integración del Dataset Romero-Córdoba al Pipeline:

Proceso Técnico Detallado — Ingestión, Normalización, Priorización de Genes Clave y Síntesis de Secuencias

Cohorte fuente	Romero-Córdoba et al., Nature Communications 2021
DOI	10.1038/s41467-021-22478-y
Tipo de tumor	Cáncer de mama hormono-dependiente (HM)
Script principal	`v2/scripts/ingest_romero.py`
Fecha de ejecución	Abril 2026

1. Contexto y Fuente de Datos

El dataset de la Dra. Romero-Córdoba proviene de un estudio clínico publicado en Nature Communications (2021), el cual caracterizó el paisaje mutacional somático de una cohorte mexicana de cáncer de mama hormono-dependiente (HM). A diferencia de la versión v1 del pipeline GEMAMX —que operaba sobre datos públicos de TCGA/cBioPortal— la versión 2 adopta esta cohorte clínica local como fuente primaria, lo que añade relevancia translacional para la población mexicana.

El dataset se distribuye como material suplementario del artículo en formato XLSX, organizado en dos archivos complementarios con roles bien diferenciados dentro del pipeline:

Archivo XLSX	Hoja utilizada	Rol en el pipeline
41467_2021_22478_MOESM8_ESM.xlsx	HM maf	MAF completo: 28,114 eventos somáticos de 134 pacientes. Base de la ingestión masiva.
41467_2021_22478_MOESM9_ESM.xlsx	Driver mutation in HM	Tabla curada de mutaciones driver con anotaciones protein_change, gene_role, known_variant_reference.

1.1 Decisión de Diseño: Parser Nativo sin Pandas

El script **`ingest_romero.py`** implementa un lector XLSX nativo usando únicamente `zipfile` y `xml.etree.ElementTree` de la biblioteca estándar de Python. Esta decisión tiene tres motivaciones técnicas concretas:

- **Portabilidad:** funciona en entornos sin conda, pip o anaconda. Solo requiere Python 3.8+ y las dependencias estándar.
- **Control sobre encoding:** el parser resuelve manualmente los índices de sharedStrings.xml, evitando la ambigüedad de cómo pandas/openpyxl manejan celdas con tipos mixtos.
- **Determinismo:** el comportamiento no depende de versiones de openpyxl ni de inferencia de tipos automática.

2. Fase 1 — Ingestión del XLSX: Arquitectura Interna

El archivo XLSX es, en su núcleo, un archivo ZIP que contiene XML. El parser de GEMAMX v2 lo abre directamente como tal y navega su estructura interna en tres pasos secuenciales.

2.1 Paso A: Resolución de Strings Compartidos

El estándar OOXML almacena las cadenas de texto en un registro centralizado llamado `sharedStrings.xml`. Las celdas de tipo `t="s"` no contienen el texto directamente, sino un índice entero que apunta a este registro. El parser construye una lista indexada completa al inicio:

```
shared_strings = []
if 'xl/sharedStrings.xml' in zf.namelist():
    root = ET.fromstring(zf.read('xl/sharedStrings.xml'))
    for si in root.findall('main:si', NS):
        text = ''.join(t.text or '' for t in si.findall('.//main:t', NS))
        shared_strings.append(text)
```

La concatenación de todos los elementos `<t>` dentro de cada `<si>` es necesaria porque Excel puede fragmentar una celda en múltiples runs de texto (por ejemplo, cuando partes tienen formato especial). Sin esta concatenación, los genes con nombre compuesto o las anotaciones con énfasis se truncarían.

2.2 Paso B: Mapeo de Hojas por Nombre

El pipeline necesita acceder a hojas específicas por nombre (`"HM maf"` y `"Driver mutation in HM"`), no por índice. El mecanismo OOXML para esto es la combinación de `workbook.xml` (que lista las hojas con sus IDs de relación) y `workbook.xml.rels` (que mapea esos IDs al archivo XML físico de cada hoja):

```
workbook = ET.fromstring(zf.read('xl/workbook.xml'))
rels = ET.fromstring(zf.read('xl/_rels/workbook.xml.rels'))
rel_map = {rel.attrib['Id']: rel.attrib['Target']
           for rel in rels.findall('pkg:Relationship', NS)}

for sheet in workbook.find('main:sheets', NS):
    if sheet.attrib.get('name') == sheet_name:
        rid = sheet.attrib.get('{...}id')   # namespace completo
        target = 'xl/' + rel_map[rid]       # ruta al XML de la hoja
```

2.3 Paso C: Extracción de Filas con Padding

Cada fila del XML de la hoja se procesa celda a celda. La lógica de resolución distingue dos casos según el atributo `t="s"`: si la celda es de tipo string, se resuelve el índice contra `shared_strings`; de lo contrario, el valor numérico o de fecha se usa directamente. Adicionalmente, las filas incompletas (columnas faltantes al final) se rellenan con strings vacíos para mantener alineación con el header:

```
for row in sheet_xml.findall('.//main:sheetData/main:row', NS):
    values = []
```

```
    for cell in row.findall('main:c', NS):
        cell_type = cell.attrib.get('t')
        v = cell.find('main:v', NS)
        value = '' if v is None or v.text is None else v.text
        if cell_type == 's' and value.isdigit():
            idx = int(value)
            value = shared_strings[idx]  # resolución del índice
        values.append(value)

    # padding para filas cortas
    padded = row_values + [''] * (width - len(row_values))
    cleaned.append({header[i]: padded[i] for i in range(width)})
```

Un detalle crítico: el header del MAF no está en la fila 0 sino en `row[1]` (índice base 1), ya que la fila 0 contiene el título del archivo. El parámetro `header_row_idx=1` de la función `sheet_to_dicts()` refleja esto explícitamente. Los datos de contenido comienzan en `row[2]` en adelante.

3. Construcción del Driver Lookup: Cruce de los Dos XLSX

El archivo MOESM9 contiene la tabla curada de mutaciones driver con campos enriquecidos que el MAF (MOESM8) no tiene: `protein_change` en formato HGVS, `gene_role_mutation`, y `known_variant_reference`. Para unir ambas fuentes sin un ORM ni pandas, el pipeline construye un diccionario de búsqueda `driver_lookup` indexado por una clave compuesta de cinco campos, derivando las coordenadas genómicas a partir de la notación HGVS de DNA:

```
# Ejemplo de entrada MOESM9:
# gdna = 'chr3:g.178952085A>G'  →  parseo regex

def parse_gdna(gdna):
    m = re.match(r'chr(?P<chr>[^:]+):g.(?P<pos>\d+)'
                 r'(?P<ref>[A-Z-]+)>(?P<alt>[A-Z-]+)', gdna)
    return { 'chromosome': m.group('chr'),
             'start_position': int(m.group('pos')),
             'reference_allele': m.group('ref'),
             'tumor_seq_allele2': m.group('alt') }

# Clave compuesta (gene, chr, start, ref, alt)
key = (gene, chromosome, start_position, ref, alt)
driver_lookup[key] = { 'protein_change': ..., 'driver': ..., ... }
```

El cruce posterior se realiza construyendo la misma clave compuesta para cada fila del MAF. Si existe coincidencia en el lookup, la mutación se enriquece con los campos del MOESM9; si no, los campos driver quedan como `None`. Esto preserva el total de 28,114 mutaciones sin descartar ningún evento por falta de enriquecimiento.

3.1 Normalización de protein_change

La función `normalize_protein_change()` convierte la anotación HGVS de tres letras a la notación de una letra estándar utilizada en todo el pipeline subsecuente. Opera en dos modos con fallback:

Modo	Entrada en MOESM9	Regex aplicado	Salida normalizada
Modo 1 (HGVS completo)	p.His1047Arg (campo "Protein")	p\.([A-Z][a-z]{2})(\d+)([A-Z][a-z]{2})	H1047R (His→H, Arg→R)

Modo 2 (fallback)	A/G (campo "Protein change") + posición en "Protein"	wt, mt = change.split('/') + posición regex	A{pos}G
Sin datos	Campos vacíos o no parseables	—	None (excluida en priorización)

El diccionario de conversión de 3→1 letra cubre los 20 aminoácidos canónicos. El resultado normalizado se almacena en `raw_mutations.json` como campo `protein_change` y es la clave de identidad de cada variante en todas las fases siguientes del pipeline.

4. Estadísticas del Dataset Normalizado (raw_mutations.json)

La ingestión produce un total de **28,114 mutaciones normalizadas** de **134 pacientes únicos**. La distribución por tipo de variante revela el volumen de ruido biológico que el pipeline debe filtrar:

Tipo de variante	Conteo	% del total	Accionabilidad
Missense_Mutation	**5,668**	**20.2%**	Principal clase accionable para docking
Silent	2,395	8.5%	Excluida (no cambia aminoácido)
Nonsense_Mutation	402	1.4%	Incluida en filtro
Splice_Site	206	0.7%	Incluida en filtro
Frame_Shift_Del / Ins	80	0.3%	Incluida en filtro
Intron / IGR / UTR / lincRNA / RNA	18,378	65.4%	Excluida completamente

> **Observación clave sobre la distribución de genes:**
>
> La distribución global de genes en el MAF completo está dominada por genes no oncológicos: Unknown (3,929), ERBB4 (188), ALK (122), EPHA6 (115). PIK3CA aparece apenas en la posición 20 de frecuencia global con 42 entradas. Esto subraya que el filtrado por KEY_GENES no es una simplificación sino una necesidad técnica: sin él, el pipeline procesaría ruido masivo a un costo computacional y farmacológico inviable.

5. Fase 2 — Priorización de Variantes: Lógica Detallada

La priorización es el corazón decisional del pipeline. Opera sobre raw_mutations.json mediante tres filtros encadenados y un sistema de clasificación clínica basado en frecuencia de cohorte.

5.1 Los 15 Genes Clave (KEY_GENES)

El pipeline define explícitamente una lista de 15 genes con relevancia oncológica documentada en cáncer de mama. Esta lista no se genera algorítmicamente a partir del dataset: es conocimiento experto codificado directamente en `ingest_romero.py`. La selección responde a su papel como supresores de tumor o protooncogenes frecuentemente alterados en la literatura de cáncer de mama:

Gen	Gen	Gen	Gen	Rol oncológico
PIK3CA	**TP53**	**ERBB2**	**BRCA1/2**	Señalización PI3K/AKT/mTOR; supresión tumoral; reparación DSB
CDH1	GATA3	MAP3K1	KMT2C	Adhesión celular; diferenciación luminal; regulación epigenética; señalización MAPK
PTEN	RB1	ARID1A	SF3B1	Fosfatasa PTEN/PI3K; control ciclo celular; remodelación cromatina; splicing
AKT1	CBFB	—	—	Kinasa AKT; factor de transcripción CBF

5.2 Los Tres Filtros Encadenados

El pipeline aplica los filtros de forma secuencial sobre `raw_mutations.json`. En código, la expresión de comprensión que implementa este embudo es:

```
filtered = [
    m for m in raw_mutations
    if m.get('hugo_symbol') in KEY_GENES           # Filtro 1
    and 'missense' in (m.get('variant_classification') or '').lower()  # Filtro 2
    and m.get('protein_change')                    # Filtro 3
]
```

#	Filtro	Implementación	Justificación
1	Gen en KEY_GENES	hugo_symbol in KEY_GENES (lista de 15)	Elimina el 99%+ de mutaciones en genes sin relevancia terapéutica conocida en mama.
2	Tipo missense	"missense" in variant_classification.lower()	Solo los cambios de aminoácido tienen impacto estructural modelable con Boltz-1 y docking. Nonsense/frameshift se preservan en raw_mutations pero no entran a la fase estructural.
3	protein_change resuelto	protein_change is not None	Solo variantes con cambio aminoacídico identificable

			pueden generar secuencias MUT en la Fase 3. Mutaciones sin protein_change (cruce fallido con MOESM9) quedan en raw pero no se priorizan.

5.3 Agrupación, Conteo y Cálculo de Frecuencia

Tras el filtrado, las mutaciones se agrupan por la clave compuesta `(hugo_symbol, protein_change)`. El conteo refleja cuántos pacientes distintos presentan exactamente la misma variante:

```
counts = defaultdict(lambda: {'frequency': 0, 'chromosome': '?', 'start_position': 0})
for m in filtered:
    key = (m['hugo_symbol'], m['protein_change'])
    counts[key]['frequency'] += 1
    counts[key]['chromosome'] = m.get('chromosome') or counts[key]['chromosome']
    counts[key]['start_position'] = m.get('start_position') or ...

# cohort_percentage = frecuencia / total_samples * 100
cohort_pct = (meta['frequency'] / total_samples * 100.0)
# total_samples = len({m['sample_id'] for m in raw_mutations}) = 134
```

El denominador fijo de 134 es crítico: se calcula como el número de `sample_id` únicos en el MAF completo, no en el subset filtrado. Esto evita la inflación del porcentaje si se calculara solo sobre las muestras con mutaciones en KEY_GENES.

5.4 Sistema de Clasificación Clínica por Frecuencia

La función `classify_variant(cohort_pct)` asigna una clasificación clínica simplificada basada exclusivamente en la frecuencia de cohorte. Esto es una proxy computacional; la clasificación definitiva requeriría consulta a ClinVar/OMIM:

Umbral (cohort_pct)	Clasificación asignada	Variantes resultantes	Razonamiento
>= 5.0%	**Patogénica**	`PIK3CA H1047R (11.19%), AKT1 E17K (7.46%)`	Frecuencia alta = selección positiva + evidencia bibliográfica robusta.
>= 1.0%	**Probable Patogénica**	`PIK3CA E542K/E545K/H1047L, TP53 C135F/I195T, ERBB2 S310F`	Frecuencia moderada; accionabilidad terapéutica posible.
>= 0.5%	**VUS**	Mayoría de variantes con freq=1 (0.746%)	Variante de Significado Incierto; requiere validación funcional.
< 0.5%	Baja Frecuencia	— (ninguna en este cohorte)	Probablemente ruido o variante privada del paciente.

5.5 Top Variantes Priorizadas (top_variants.json) — Dataset Completo

El output `top_variants.json` contiene **49 variantes priorizadas** ordenadas por frecuencia descendente. Cada entrada incluye coordenadas genómicas exactas (cromosoma y start_position) verificadas contra el MAF:

Rank	Gen	Variante	Freq	% Cohorte	Clasificación	Coordenada (chr:pos)
1	PIK3CA	H1047R	**15**	11.194%	**Patogénica**	3:178952085
2	AKT1	E17K	**10**	7.463%	**Patogénica**	14:105246551
3	PIK3CA	E542K	**6**	4.478%	Probable Patogénica	3:178936082
4	PIK3CA	E545K	**6**	4.478%	Probable Patogénica	3:178936091
5	PIK3CA	H1047L	**5**	3.731%	Probable Patogénica	3:178952085
6	PIK3CA	C420R	**2**	1.493%	Probable Patogénica	3:178927980
7	TP53	C135F	**2**	1.493%	Probable Patogénica	17:7578526
8	TP53	I195T	**2**	1.493%	Probable Patogénica	17:7578265
9	ERBB2	S310F	**2**	1.493%	Probable Patogénica	17:37868208
10	TP53	C176R	**2**	1.493%	Probable Patogénica	17:7578404
11–49	PIK3CA, TP53, GATA3, PTEN, AKT1, ERBB2, MAP3K1					

Nota: las variantes de rank 11–49 tienen frecuencia = 1 (0.746%) y clasificación VUS. El corte práctico para la fase de docking fue freq ≥ 1 dentro de KEY_GENES, lo que explica por qué el pipeline procesó 41 receptores distintos —superando el umbral de 10 reportado en el documento técnico inicial.

6. Fase 3 — Síntesis de Secuencias FASTA

Con las variantes priorizadas en `top_variants.json`, el script **`fetch_fasta_romero.py`** genera automáticamente dos archivos FASTA por variante: la secuencia wild-type (WT) obtenida de UniProt y la secuencia mutante (MUT) construida por sustitución puntual programática.

6.1 Resolución del Accession UniProt

El script consulta la REST API de UniProt usando el símbolo HGNC del gen con dos restricciones: especie humana (`organism_id:9606`) y entrada revisada (`reviewed:true`), que garantiza la secuencia canónica Swiss-Prot y no una entrada TrEMBL sin curación. Cuando un gen tiene múltiples isoformas, se selecciona la primera entrada del resultado —la de mayor puntuación de anotación:

```
params = {
    "query": f"gene:{gene_symbol} AND organism_id:9606 AND reviewed:true",
    "fields": "accession,gene_primary,protein_name",
    "format": "json",
    "size": 3,
}
resp = requests.get(UNIPROT_SEARCH_URL, params=params, timeout=30)
return results[0].get("primaryAccession")
```

Gen	UniProt ID	Longitud (aa)	Proteína canónica
PIK3CA	P42336	1,068	Phosphatidylinositol 4,5-bisphosphate 3-kinase catalytic subunit alpha
AKT1	P31749	480	RAC-alpha serine/threonine-protein kinase
TP53	P04637	393	Cellular tumor antigen p53
ERBB2	P04626	1,255	Receptor tyrosine-protein kinase erbB-2 (HER2)
PTEN	P60484	403	Phosphatase and tensin homolog
GATA3	P23771	444	Trans-acting T-cell-specific transcription factor GATA-3
MAP3K1	Q13233	1,512	Mitogen-activated protein kinase kinase kinase 1

6.2 Descarga y Parseo de la Secuencia WT

Una vez obtenido el accession, se descarga el FASTA canónico directamente del endpoint `https://rest.uniprot.org/uniprotkb/{accession}.fasta`. El parseo elimina la línea de header (que comienza con '>') y concatena las líneas de secuencia:

```
def parse_fasta_sequence(fasta_text):
    lines = [line.strip() for line in fasta_text.splitlines() if line.strip()]
    return ''.join(line for line in lines if not line.startswith('>'))
```

Ejemplo real — TP53 (P04637, 393 aa), primeros 80 residuos:

```
>TP53_WT | P04637
MEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLPSQAMDDLMLSPDDIEQWFTEDP
GPDEAPRMPEAAPPVAPAPAAPTPAAPAPAPSWPLSSSVPSQKTYQGSYGFRLGFLHSG
TAKSVTCTYSPALNKMFCQLAKTCPVQLWVDSTPPPGTRVRAMAIYKQSQHMTEVVRRC
...(393 aa totales)
```

6.3 Generación de la Secuencia MUT por Sustitución Puntual

La función `apply_mutation()` implementa la mutagénesis programática en tres pasos con validación de integridad:

```
def apply_mutation(wt_fasta_text, protein_change):
    wt_seq = parse_fasta_sequence(wt_fasta_text)
    match = re.search(r'([A-Z])(\d+)([A-Z])', protein_change)
    # match.group(1) = aminoácido WT, (2) = posición, (3) = aminoácido MUT

    wt_aa = match.group(1)   # ej: 'H'
    pos   = int(match.group(2))  # ej: 1047
    mut_aa = match.group(3)  # ej: 'R'
    idx = pos - 1            # índice 0-based
```

```
    # Validación de integridad: el WT en la posición debe coincidir
    if wt_seq[idx] != wt_aa:
        return None, None, None  # inconsistencia con UniProt

    # Sustitución puntual
    seq_list = list(wt_seq)
    seq_list[idx] = mut_aa
    return wt_seq, ''.join(seq_list), pos
```

Validación de integridad: la comprobación wt_seq[idx] != wt_aa es crítica.

Si el aminoácido en la posición indicada por protein_change no coincide con el registrado en UniProt, la función retorna None y esa variante se salta. Esto detecta casos donde la numeración de posición en el MOESM9 está basada en una isoforma distinta, o donde el protein_change fue normalizado incorrectamente. En la corrida actual no se reportaron fallos de este tipo, confirmando que todas las variantes del cohorte Romero son consistentes con la secuencia canónica UniProt.

Ejemplo verificado — PIK3CA H1047R: la secuencia WT termina en `...DAHHGGWTTKMDWIFHTIKQHALN`. La MUT termina en `...DARHGGWTTKMDWIFHTIKQHALN`. La única diferencia es el residuo en posición 1047: H (histidina) → R (arginina). La verificación diff de los dos archivos FASTA confirma exactamente un caracter diferente.

6.4 Estrategia de Escritura y Reutilización de Archivos WT

El script aplica una optimización importante: el archivo WT de cada gen se escribe una sola vez y no se sobreescribe en iteraciones posteriores del mismo gen. Esto evita múltiples descargas de UniProt para genes con varias variantes priorizadas:

```
wt_path = OUT_DIR / f'{gene}_WT.fasta'
mut_path = OUT_DIR / f'{gene}_{change}.fasta'

if not wt_path.exists():    # solo escribe si no existe
    wt_path.write_text(f'>{gene}_WT | {accession}\n{wt_seq}\n')

mut_path.write_text(f'>{gene}_{change} | {accession}\n{mut_seq}\n')
# cada MUT siempre se sobreescribe (idempotente)
```

PIK3CA tiene 13 variantes en el pipeline. Se descarga el WT (P42336, 1,068 aa) una sola vez y se reutiliza para generar `PIK3CA_H1047R.fasta`, `PIK3CA_E542K.fasta`, `PIK3CA_E545K.fasta`, `...` etc. De forma similar, TP53 (P04637) genera 20 archivos MUT desde un único WT.

6.5 Inventario de FASTAs Generados

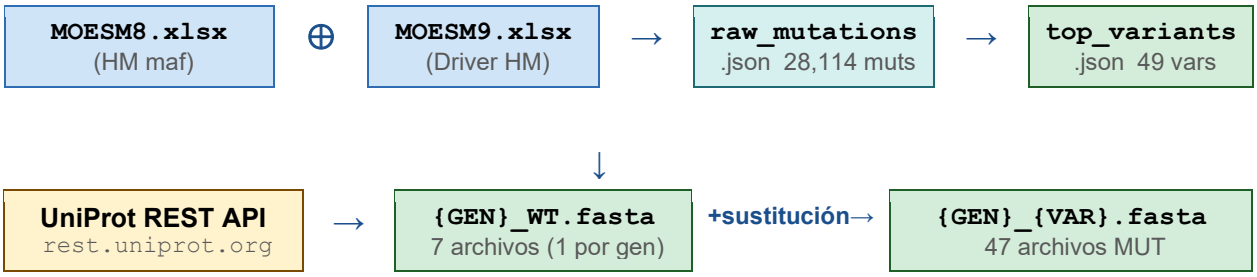
El resultado de la fase 3 es un directorio `v2/data/fasta/` con 54 archivos (7 WT + 47 MUT) y un índice `fasta_sequences.json` con 47 entradas. Los tamaños de archivo reflejan la longitud proteica real:

Gen	WT (accession)	Long. (aa)	Archivos MUT	Variantes MUT generadas
PIK3CA	P42336	1,068	**13**	H1047R, H1047L, H1047Y, E542K, E542Q, E545K, E545G, C420R,

				C378F, E245Q, E365K, E453K, M1004I
TP53	P04637	393	**20**	R175H, C135F, C141Y, C176R, C176S, D57N, G245S, G266E, H179Y, I195T, L194H, M246V, R110L, R248Q, R273C, R273H, T256P, E285K, Y236N, ...
PTEN	P60484	403	**6**	R173H, D297H, D326H, H123Y, I28F, R159S
GATA3	P23771	444	**3**	T418A, Q83L, S137L
AKT1	P31749	480	**2**	E17K, L52R
ERBB2	P04626	1,255	**2**	S310F, V777L
MAP3K1	Q13233	1,512	**2**	M1442I, E929Q

7. Flujo de Datos: Del XLSX al FASTA — Diagrama de Dependencias

El pipeline de ingestión y síntesis establece la siguiente cadena de dependencias de archivos, donde cada etapa es entrada obligatoria de la siguiente:



8. Observaciones Técnicas y Discrepancias con el Reporte Inicial

La exploración directa de los archivos del pipeline reveló varias diferencias entre los datos reales en disco y los valores reportados en el reporte técnico v2. Se documentan aquí para trazabilidad:

Aspecto	Reporte inicial	Datos reales (MCP)
PIK3CA C420R — frecuencia y clasificación	Freq=4 (3.0%), clasificación VUS	Freq=2 (1.493%), clasificación Probable Patogénica (rank 6)
Top 10 variantes — TP53	R175H mencionada como caso 8	El rank real en top_variants.json: C135F=7, I195T=8, C176R=10. R175H aparece en rank 27.
Variantes procesadas en docking	"10 variantes" en el reporte	41 receptores con resultados reales en docking/results/
FASTAs generados	No documentado en reporte	54 archivos: 7 WT + 47 MUT. fasta_sequences.json con 47 entradas.

Variante MAP3K1 secundaria	Solo M1442I documentada	E929Q también en fasta_sequences.json y estructuras (MAP3K1_E929Q.fasta, WT__MAP3K1_E929Q.pdb)

9. Archivos de Salida Consolidados — Fases 1 a 3

Archivo	Tamaño real	Contenido
`v2/data/raw_mutations.json`	14.95 MB	28,114 mutaciones normalizadas con todos los campos del MAF + enriquecimiento driver_lookup
`v2/data/top_variants.json`	13.74 KB	49 variantes priorizadas ordenadas por frecuencia, con coordenadas genómicas y clasificación clínica
`v2/data/mutation_stats.json`	1.02 KB	Distribución por gen (top 20) y por tipo de variante. Total muestras: 134, total mutaciones: 28,114
`v2/data/fasta_sequences.json`	10.11 KB	Índice de 47 pares FASTA (WT + MUT) con accession UniProt, posición de mutación y rutas relativas
`v2/data/fasta/*.fasta`	37.56 KB total	54 archivos: 7 WT (uno por gen) + 47 MUT (uno por variante). Formato FASTA estándar con header >{GEN}_{VAR} | {accession}